

Relatório Acadêmico - ADO 2: Física Aplicada

Curso: Ciência da Computação | Disciplina: Prática Extensionista - Física Aplicada

1. Equações Utilizadas e Lógica de Implementação (Sistema 3.1)

Derivação da aceleração (a) do Plano Inclinado:

Com base no algoritmo atualizado, a lógica foi condensada utilizando as variáveis dinâmicas *Massa_1*, *Massa_2* e *Angulo*. A Força Normal (N) e a Força Motriz foram definidas vetorialmente como:

$$N = \text{Massa_1} * G * \cos(\theta)$$

$$F_{\text{motriz}} = \text{Massa_2} * G - \text{Massa_1} * G * \sin(\theta)$$

O script utiliza um operador ternário para definir o estado de movimento. A variável *Movimento_Uniforme* atua como o coeficiente de atrito (μ):

a = 0 se $|F_{\text{motriz}}| \leq \text{Movimento_Uniforme} * N$, caso contrário:

$$a = [F_{\text{motriz}} - \text{Movimento_Uniforme} * N * \text{sin}(\theta)] / (\text{Massa_1} + \text{Massa_2})$$

As grandezas cinemáticas seguem a integração direta no tempo do MRUV: Posição = $0.5 * a * t^2$ e Velocidade = $a * t$.

Fórmulas Fechadas - Sistemas 3.2 e 3.3:

- **Força Centrípeta (MCU):** $\omega = \text{Velocidade} / \text{Raio}$ | $x(t) = \text{Raio} * \cos(\omega t)$ | $y(t) = \text{Raio} * \sin(\omega t)$ | $F_c = \text{Massa} * \text{Velocidade}^2 / \text{Raio}$
- **Arrasto Linear:** $v_{\text{term}} = (\text{Massa} * G) / \text{Coeficiente}$ | $v(t) = v_{\text{term}} * (1 - e^{-t / (\text{Massa} / \text{Coeficiente})})$
- **Arrasto Quadrático:** $v_{\text{term}} = \sqrt{(\text{Massa} * G) / \text{Coeficiente}}$ | $v(t) = v_{\text{term}} * \tanh((G * t) / v_{\text{term}})$

2. Determinação do Ponto Crítico de Atrito

Fixando os parâmetros padrão da simulação (*Massa_1* = 5.0, *Massa_2* = 7.0, *Angulo* = 30°):

$$F_{\text{motriz}} = 7.0 * 9.81 - 5.0 * 9.81 * \sin(30^\circ) = 68.67 - 24.525 = 44.145 \text{ N.}$$

$$\text{Força Normal (N)} = 5.0 * 9.81 * \cos(30^\circ) = 42.48 \text{ N.}$$

O sistema entra em repouso quando a condição do operador ternário é satisfeita ($|F_{\text{motriz}}| \leq \text{Movimento_Uniforme} * N$). Logo, o valor crítico para o slider *Movimento_Uniforme* é calculado por:

$$\text{Movimento_Uniforme_critico} = 44.145 / 42.48 \approx 1.039$$

Na interface gráfica da aplicação, ao deslizar *Movimento_Uniforme* para um valor maior que 1.04, a aceleração exibida na tela zera automaticamente, validando perfeitamente a lógica computacional implementada.

3. Comparação da Velocidade Terminal no Arrasto

O algoritmo calcula a velocidade terminal de duas formas distintas, dependendo do seletor da interface:

- **Linear:** O corpo sofre desaceleração estritamente proporcional à velocidade, utilizando a função exponencial do NumPy (np.exp). A curva apresenta uma transição mais suave desde o início do movimento.
- **Quadrático:** A resistência escala com o quadrado da velocidade, modelada pela tangente hiperbólica (np.tanh). A queda inicial acompanha o comportamento de queda livre ($G * t$), mas sofre uma saturação muito mais abrupta ao se aproximar de v_{term} .

A renderização gráfica através do Matplotlib confirma visualmente que ambas as funções assintóticas estabilizam na linha tracejada (v_{term}), com distanciamento claro em relação à queda livre ideal.

4. Aplicações Práticas e Modelagem Computacional

1. **Plano Inclinado:** A lógica do operador ternário emula sistemas automatizados de pontes rolantes e elevadores industriais, garantindo que os motores apenas atuem se a tração vencer a zona de atrito estático.
2. **Força Centrípeta:** As equações de trajetória (x , y) aplicadas sobre o vetor de tempo simulam a dinâmica de satélites e são os mesmos cálculos vetoriais usados em motores de física (Game Engines) para estabilidade direcional.
3. **Força de Arrasto:** O seletor linear/quadrático reflete a modelagem aerodinâmica, permitindo diferenciar fluidos densos e viscosos (comportamento linear) do impacto do ar em alta velocidade (comportamento quadrático).